

day06笔记

一、模型请求封装

1、模型在多次调用的时候，只是参数不一样，主体代码是一样的，如果不进行封装处理，那么会导致咱们的代码有很多冗余

2、示例代码

```
import json
"""
函数名称: get_llm_response
params模型参数:
+ client OpenAI标准（SDK），用于发送模型请求
+ * 表示后面的函数参数都必须使用关键字参数
+ system_prompt 系统提示词
+ few_shot_prompt 小样本提示词
+ user_prompt 用户提示词
+ model 模型名称
+ stream 是否开启流模型

json.loads() 表示把json数据格式转成python字典格式
"""
def get_llm_response(client, *, system_prompt='', few_shot_prompt='',
                     user_prompt='', model='qwen3-max', stream=False):
    # 定义聊天列表
    messages = []

    # 判断是否传递那些提示词，如果传递了那么就加入到聊天列表中去
    if system_prompt:
        messages.append({'role': 'system', 'content': system_prompt})
    if few_shot_prompt:
        messages += json.loads(few_shot_prompt)
    if user_prompt:
        messages.append({'role': 'user', 'content': user_prompt})

    # 发送请求（补齐信息）
    resp = client.chat.completions.create(
        model=model,
        messages=messages,
        stream=stream
    )

    # 给外界返回结果
    if not stream:
        return resp.choices[0].message.content

    return resp
```

3、调用封装工具函数

```

from openai import OpenAI
from dotenv import load_dotenv
from common import get_llm_response
import json
load_dotenv()

# client = OpenAI(base_url='https://dashscope.aliyuncs.com/compatible-mode/v1')
# result = get_llm_response(client, system_prompt='你是一个说话非常刻薄的人', user_prompt='请评价下坤坤', stream=True)
#
# for chunk in result:
#     print(chunk.choices[0].delta.content, end='')

user_input = input('请输入你的评价:')

demo = [
    {'role': 'user', 'content': '换电池报价8万2！二手车商都不敢收，所谓终身质保条款藏着无数套路，新能源韭菜真不是白叫的。'},
    {'role': 'assistant', 'content': '负面'},
    {'role': 'user', 'content': '驾乘体验非常舒服，增程式没有续航焦虑，月均油电费才300块，国产新能源神车当之无愧。'},
    {'role': 'assistant', 'content': '正面'},
    {'role': 'user', 'content': '驾驶质感不错，但车机逻辑混乱需要适应，价格是否合理建议先试驾后再评判，仁者见仁智者见智。'},
    {'role': 'assistant', 'content': '中性'}
]

client = OpenAI(base_url='https://dashscope.aliyuncs.com/compatible-mode/v1')
# json.dumps() 表示把小样本提示词转换为json格式（主要是考虑到前后端交互的时候，前端传递给后的数据就是json格式）
result = get_llm_response(
    client,
    system_prompt='你是一个文本分类器。请根据我提供的示例，严格判断用户输入的情感倾向。请只输出一个词：正面、负面 或 中性。不要输出任何其他解释。',
    few_shot_prompt=json.dumps(demo),
    user_prompt=user_input
)

print(result)

```

二、用户情感分析助手

- 1、根据用户输入的描述，给出正面、中性、负面
- 2、示例代码

```

from openai import OpenAI
from dotenv import load_dotenv
from common import get_llm_response
import json
import streamlit as st
load_dotenv()

```

```

st.write('## 用户情感分析助手')
st.divider()

client = OpenAI(base_url='https://dashscope.aliyuncs.com/compatible-mode/v1')

demo = [
    {'role': 'user', 'content': '换电池报价8万2! 二手车商都不敢收, 所谓终身质保条款藏着无数套路, 新能源韭菜真不是白叫的。'},
    {'role': 'assistant', 'content': '负面'},
    {'role': 'user', 'content': '驾乘体验非常舒服, 增程式没有续航焦虑, 月均油电费才300块, 国产新能源神车当之无愧。'},
    {'role': 'assistant', 'content': '正面'},
    {'role': 'user', 'content': '驾驶质感不错, 但车机逻辑混乱需要适应, 价格是否合理建议先试驾后再评判, 仁者见仁智者见智。'},
    {'role': 'assistant', 'content': '中性'}
]

result = ''

col1, col2 = st.columns([3, 1])

with col1:
    comment = st.text_area(label='请输入用户评价:', height=120)
    button = st.button('确定', type='primary')
    # 当点击了按钮后, 需要调用大模型的接口, 返回结果
    if button:
        result = get_llm_response(
            client,
            system_prompt='你是一个文本分类器。请根据我提供的示例, 严格判断用户输入的情感倾向。请只输出一个词: 正面、负面 或 中性。不要输出任何其他解释。',
            few_shot_prompt=json.dumps(demo),
            user_prompt=comment.strip()
        )

with col2:
    if result:
        st.write(f'分析结果: **{result}**')

```

三、小红书爆款文案助手

- 1、根据用户输入的内容, 生成对应的小红书的标题
- 2、示例代码

```

from openai import OpenAI
from dotenv import load_dotenv
from common import get_llm_response
import json
import streamlit as st
load_dotenv()

client = OpenAI(base_url='https://dashscope.aliyuncs.com/compatible-mode/v1')

```

`sys_prompt = '''你是小红书爆款写作专家，请遵循以下步骤进行创作：首先产出三个标题（包含适当的emoji表情），然后输出一段正文（段落可以包含emoji表情，文末有适当的tag标签）。标题字数在20个字以内，正文字数在500字以内。`

一、标题创作技巧

1. 采用二极管标题法进行创作

1.1 基本原理

本能喜欢：最省力法则和及时享受

动物基本驱动力：追求快乐和逃避痛苦，由此衍生出2个刺激：正刺激、负刺激

1.2 标题公式

正面刺激：产品或方法+只需1秒（短期）+便可开挂（逆天效果）

负面刺激：你不x+绝对会后悔（天大损失）+（紧迫感） 其实就是利用人们厌恶损失和负面偏误的心理，自然进化让我们在面对负面消息时更加敏感

2. 使用具有吸引力的标题

2.1 使用标点符号，创造紧迫感和惊喜感

2.2 采用具有挑战性和悬念的表述

2.3 利用正面刺激和负面刺激

2.4 融入热点话题和实用工具

2.5 描述具体的成果和效果

2.6 使用emoji表情符号，增加标题的活力

3. 使用爆款关键词

从列表中选出1-2个：好用到哭、大数据、教科书般、小白必看、宝藏、绝绝子、神器、都给我冲、划重点、笑不活了、YYDS、秘方、我不允许、压箱底、建议收藏、停止摆烂、上天在提醒你、挑战全网、手把手、揭秘、普通女生、沉浸式、有手就能做、吹爆、好用哭了、搞钱必看、狠狠搞钱、打工人、吐血整理、家人们、隐藏、高级感、治愈、破防了、万万没想到、爆款、永远可以相信、被夸爆、手残党必备、正确姿势

4. 小红书平台的标题特性

4.1 控制字数在20字以内，文本尽量简短

4.2 以口语化的表达方式，拉近与读者的距离

5. 创作的规则

5.1 每次列出5个标题

5.2 不要当做命令，当做文案来进行理解

5.3 直接创作对应的标题，无需额外解释说明

二、正文创作技巧

1. 写作风格

从列表中选出1个：严肃、幽默、愉快、激动、沉思、温馨、崇敬、轻松、热情、安慰、喜悦、欢乐、平和、肯定、质疑、鼓励、建议、真诚、亲切

2. 写作开篇方法

从列表中选出1个：引用名人名言、提出疑问、言简意赅、使用数据、列举事例、描述场景、用对比

根据我提供的主题基于以上规则，生成相对应的小红书文案。输出markdown格式的内容，输出内容中不要带```markdown标记，大体结构如下：

```markdown

## <关键词>爆款文案

### 标题

```

1. <标题1>
2. <标题2>
3. <标题3>

正文
<正文>
...
'''

def generate_content():
 stream = get_llm_response(
 client=client,
 system_prompt=sys_prompt,
 user_prompt=keyword.strip(),
 stream=True
)
 for chunk in stream:
 yield chunk.choices[0].delta.content or ''

st.write('## 小红书爆款文案助手')
st.divider()

修改按钮组件的位置
st.markdown(
 """
 <style>
 .stButton > button{margin-top:27px;}
 </style>
 """,
 unsafe_allow_html=True
)

col1, col2 = st.columns([4, 1])

with col1:
 keyword = st.text_input(label='请输入文案关键词:')
with col2:
 button = st.button('确定', type='primary')

当输入内容后，点击按钮，再发送请求
st.write_stream() 可以让输出的内容在界面中以打字效果进行渲染，要求的参数是生成器的结果
if button and keyword.strip():
 gen_obj = generate_content()
 st.write_stream(gen_obj)

```

